

# I. Fundamentals of DBMS

## What is DBMS? DBMS Examples?

A **Database Management System (DBMS)** is a collection of interrelated data and various programs that are used to handle the data. The primary goal of a DBMS is to provide a way to **store and retrieve the required information** from the database in a convenient and efficient manner.

The long form of RDBMS is **Relational Database Management System**.

**Examples of RDBMS (SQL Databases):** MySQL, Oracle, MS ACCESS, Microsoft SQL Server, and Sybase.

**Examples of NoSQL Databases:** MongoDB (document store), DynamoDB, Riak, Redis (key-value store), Neo4J (graph database), and Cassandra (wide column store).

## What are the advantage of DBMS?

The advantages of using a DBMS include:

1. **Removes Data Redundancy:** There is no duplication of data in the database.
2. **Efficient Data Retrieval:** Allows the desired data to be retrieved efficiently.
3. **Data Integrity:** Constraints can be applied to data to ensure it falls within a specified range or format.
4. **Atomicity:** If an operation is performed on the database, the change must be **reflected for the entire database** (i.e., all operations succeed or none do).
5. **Concurrent Access:** The DBMS allows multiple users concurrent access using synchronization techniques.
6. **Security:** Security policies can be applied to ensure users access only the desired part of the database.

## What is redundancy, isolation & atomicity problem?

- **Redundancy Problem:** This occurs when the same piece of information appears repeatedly in various formats at several places in the system (duplication of data). This leads to problems like redundant storage, wasted space, and anomalies.
- **Atomicity Problem:** This is the difficulty in ensuring that a particular operation must be carried out **entirely or not at all**. If atomicity is not maintained, a transaction might fail partially, leaving the database in an inconsistent state (e.g., if money is withdrawn but the balance is not updated).
- **Isolation Problem (Anomaly):** Isolation is a fundamental property that guarantees concurrent transactions execute as if they were running sequentially. A problem arises when concurrent execution violates this property, leading to inconsistencies such as the **Lost Update Problem** (where one transaction's update is overwritten by another) or **Dirty Read** (where a transaction reads data written by an uncommitted transaction).

---

## II. ER Modeling and Keys

### What is attributes? Types of attributes in DBMS?

An **attribute** defines the properties of a data object or entity.

Types of attributes include:

1. **Simple Attributes:** Attributes drawn from **atomic value domains** (indivisible units), such as Age.
2. **Composite Attributes:** Attributes that consist of a hierarchy of attributes, such as Address (which may include Number, Street, and Suburb).
3. **Single-valued Attributes:** Attributes that have only one value for each entity, like a Student ID.
4. **Multivalued Attributes:** Attributes that can have a set of values for a single entity, such as the Degrees earned by a person.
5. **Derived Attributes:** Attributes whose values are calculated from other attributes, such as Age calculated from DateOfBirth.

### What is Strong entity and weak entity?

- **Strong Entity Set:** An entity set that **has its own primary key**. It is represented by a rectangle.
- **Weak Entity Set:** An entity that cannot be uniquely identified by its attributes alone and **does not have sufficient attributes to form a primary key on its own**. It is represented by a double rectangle. The primary key of a weak entity set is a combination of its partial key (discriminator) and the primary key of its identifying strong entity set.

### What are the Types of mapping cardinality?

Mapping cardinalities express the number of entities to which another entity can be associated via a relationship set [181, Q.23]. Used primarily in binary relationship sets, the types are:

1. **One to One (1:1):** An entity in A is associated with at most one entity in B, and vice versa.
2. **One to Many (1:N):** An entity in A can be associated with any number of entities in B, but an entity in B can be associated with at most one entity in A.
3. **Many to One (N:1):** An entity in A is associated with at most one entity in B, but an entity in B can be associated with any number of entities in A.
4. **Many to Many (N:N):** An entity in A is associated with any number of entities in B, and an entity in B is associated with any number of entities in A.

### What is Primary key, foreign key, candidate key?

- **Super Key (SK):** A set of one or more attributes within a table that can uniquely identify each record.

- **Candidate Key (CK):** A minimal super key; the smallest combination of attributes that uniquely identifies each record.
- **Primary Key (PK):** The specific candidate key chosen by the database designer to uniquely identify tuples in the relation. A key constraint requires that the value of the primary key must be unique and cannot be NULL.
- **Foreign Key (FK):** An attribute or set of attributes in one table (child table) that refers to the primary key of another table (parent table). The foreign key constraint enforces **referential integrity**.

### III. SQL Language and Commands

**What is long form DDL, DML, DCL, TCL. Which are DDL, DML, DCL, TCL, Commands?**

| Acronym | Long Form                         | Purpose                                                                            | Example Commands                                                                        |
|---------|-----------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| DDL     | <b>Data Definition Language</b>   | Used to specify and modify the database schema (structure of tables, views, etc.). | <code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> .                          |
| DML     | <b>Data Manipulation Language</b> | Used to access or manipulate data within the database.                             | <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> . |
| DCL     | <b>Data Control Language</b>      | Used to control access rights and authorization to data.                           | <code>GRANT</code> , <code>REVOKE</code> .                                              |
| TCL     | Transaction Control Language      | Used to manage transactions (saving or undoing changes).                           | <code>COMMIT</code> , <code>ROLLBACK</code> .                                           |

**What is the use of View, Index, Sequence, Synonym?**

- **View:** A virtual table defined by a `SELECT` statement. Views are used to present an abstract, partial view of the data or to enforce security.
  - Syntax: `CREATE VIEW name_of_view AS SELECT column1, column2, ... FROM table_name1, ... WHERE condition;`
- **Index:** A special data structure (a lookup table/pointer) that stores a small part of a table's data to provide a fast access path to records. Indexing makes data retrieval fast and efficient.
  - Syntax: `CREATE INDEX index_name ON table_name;`
- **Sequence/Synonym:** The provided source material mentions these concepts in the syllabus overview but does not provide details on their definition, usage, or syntax in the main excerpts.

**Use of alter...add, alter...delete, alter...modify command**

The `ALTER` command is a DDL command used to change the schema of a table [179, Q.19, 247].

- **ALTER TABLE ... ADD:** Used to add a new column to an existing table.
  - Syntax: `ALTER TABLE table_name ADD column_name datatype; .`
- **ALTER TABLE ... DROP COLUMN:** Used to delete a specified column from the table.
  - Syntax: `ALTER TABLE table_name DROP COLUMN column_name; .`
- **ALTER TABLE ... MODIFY:** Used to change the characteristics (like data type or constraints) of an existing column.
  - Example Use: `ALTER TABLE EMPLOYEE MODIFY Designation VARCHAR(20) NOT NULL UNIQUE; .`

### Syntax of update...set, delete command

- **UPDATE...SET Command:** Used to modify the existing records in a table.
  - Syntax: `UPDATE table_name SET col1=value1, col2=value2, ... WHERE condition; .`
- **DELETE Command:** Used to delete one or more records based on some condition.
  - Syntax (Conditional Deletion): `DELETE FROM table_name WHERE condition; .`
  - Syntax (Delete All Rows): `DELETE FROM table_name; .` (Deletes all records but keeps the table definition).

### Difference between drop and truncate command.

The sources explicitly differentiate between `DROP TABLE` and `DELETE FROM` :

- **DROP TABLE:** This command deletes all records in the table **and** also removes the table definition (schema) from the database catalog entirely.
- **DELETE FROM table\_name;:** This command deletes all rows in the table but **leaves the table definition** (schema) intact for future use.
- The term `TRUNCATE` is not explicitly defined in the provided source material, so a direct comparison is not possible.

### To display table content which command is used?

The `SELECT` statement is used to fetch data from the database table. To display all content:

- `SELECT * FROM table_name; .`

### Types of index?

Indexes can be created in SQL using `CREATE INDEX` . If specified, an index can enforce uniqueness using `CREATE UNIQUE INDEX` . Indexes can also be created on multiple columns, which is called a **Compound Index**. MongoDB uses indexing for efficient query execution, including a default primary index ( `_id` ) and secondary indexes.

## What is join & use of join? Types of it?

A **Join** clause is used to **combine records** (fields) from two or more tables in a database by matching values common to those tables.

The types of join operations are:

1. **Inner Join (EQUIJOIN):** Returns only those rows that have matching values in both tables based on the join predicate [271, 472, Q.17].
2. **Left Join (Outer Join):** Returns all rows from the left table and the matched rows from the right table (or NULL if there is no match).
3. **Right Join (Outer Join):** Returns all rows from the right table and the matched rows from the left table (or NULL if there is no match).
4. **Full Join (Outer Join):** Combines the results of both left and right outer joins, returning all records from both tables, filling in NULLs for missing matches.

## Which are the constraints in DBMS?

**Constraints** are rules or restrictions set on the database to enforce data integrity.

Key types of constraints include:

1. **Domain Constraint:** Defines the permissible values (domain) for an attribute based on its data type (e.g., string, integer, date).
2. **Key Constraint (or NULL Constraint):** Ensures unique identification of records (e.g., Primary Key values must be unique and non-NULL).
3. **Integrity Constraints:**
  - **Entity Integrity:** Requires that the value of the primary key attribute cannot be NULL.
  - **Referential Integrity:** Requires that a foreign key value in one table must reference a valid, existing primary key in the parent table.
4. **SQL Specific Constraints:** **UNIQUE** , **NOT NULL** , and **CHECK** (limits the value range).
5. **Enterprise Constraints (Semantic):** Additional rules specified by users or administrators, often spanning multiple tables (e.g., salary rules).

## Data types in SQL/oracle?

SQL supports various data types:

- **Numeric:** Integer types ( **INT** , **SMALLINT** ), Floating-point types ( **REAL** , **FLOAT** ), and Fixed-point numbers ( **DECIMAL** , **NUMERIC** ).
- **Character String:** Fixed length ( **CHAR (n)** ) and Varying length ( **VARCHAR (n)** , **VARCHAR2 (n)** ).
- **Large Object (LOB):** Character LOB ( **CLOB** ) and Binary LOB ( **BLOB** ).
- **Date/Time:** **DATE** (YYYY-MM-DD) and **TIMESTAMP** .

- **Boolean:** Values of `TRUE` , `FALSE` , or `NULL` .

## Difference between CHAR and VARCHAR?

Based on the explicit definitions provided in the query, the differences are:

- **Storage and Retrieval:** `CHAR` and `VARCHAR` types differ in how they are stored and retrieved [User Query].
- **Length:** A `CHAR` column length is fixed to the length that is declared (ranging from 1 and 255) [User Query]. `VARCHAR` is a variable-length character string.
- **Padding:** When `CHAR` values are stored, they are **right padded using spaces** to the specific length [User Query].
- **Trailing Spaces:** Trailing spaces are removed when `CHAR` values are retrieved [User Query].

## What is integrity in DBMS?

**Data Integrity** means that the data values entered into the database fall within a specified range and adhere to a specific format. Integrity constraints (like Entity Integrity and Referential Integrity) are rules applied to ensure the validity and consistency of data.

## What is the use of distinct, Like keyword?

- **DISTINCT:** Used along with the `SELECT` statement to obtain unique values and eliminate duplicate elements from the result set.
- **LIKE:** Used within the `WHERE` clause to perform pattern matching on strings. It employs wildcards like the percent symbol ( `%` ), which matches zero, one, or multiple characters, and the underscore symbol ( `_` ), which matches any single character.

## What is the use of order by, having, groupby clause?

- **ORDER BY:** Used to sort the records in the table, typically in ascending order (default) or descending order ( `DESC` ) based on specified columns.
- **GROUP BY:** Used in the `SELECT` statement to group rows that have identical values, often used in conjunction with aggregate functions to return a single row for every grouped item.
- **HAVING:** Filters records that operate on **summarized results** generated by the `GROUP BY` clause. It is mandatory for aggregate functions when filtering groups, whereas the `WHERE` clause filters individual records.

## Which are aggregate functions?

Aggregate functions perform a calculation on a set of values to return a single scalar value. The five built-in aggregate functions in SQL are:

1. **AVG** (Average)
  2. **MIN** (Minimum)
  3. **MAX** (Maximum)
  4. **SUM** (Total)
  5. **COUNT** (Count)
- 

## IV. Normalization and Transaction Management

### What is normalization? Types of it?

**Normalization** is the process of reorganizing data in a database to eliminate redundant data and ensure that all logical dependencies are met. It helps reduce the chances of data errors and improves data integrity and consistency.

The common types of Normal Forms are:

1. **First Normal Form (1NF)**: Requires all attributes to have atomic (indivisible) domains.
2. **Second Normal Form (2NF)**: Must be in 1NF and must eliminate **partial functional dependency** (where a non-prime attribute depends only on a part of the candidate key).
3. **Third Normal Form (3NF)**: Must be in 2NF and must eliminate **transitive dependency** (where a non-key attribute depends on another non-key attribute).
4. **Boyce-Codd Normal Form (BCNF)**: A stricter version of 3NF. For every non-trivial functional dependency  $X \rightarrow Y$ ,  $X$  must be a super key.

### What is Stored procedure & Stored function?

- **Stored Procedure**: A named PL/SQL subprogram or block of statements that executes a task but **does not return a value directly**. Procedures use parameters that can be defined as **IN** , **OUT** , or **IN OUT** .
- **Stored Function**: A named PL/SQL subprogram that takes parameters and is explicitly required to **return a single value** to the calling environment.

### What is cursor? Types of it? What are the attributes of cursor?

A **cursor** is a pointer to the memory area (context area) created by Oracle when an SQL statement is processed. It is used to fetch and process rows one at a time.

#### Types of Cursors:

1. **Implicit Cursor**: Created automatically by the system whenever DML operations ( **SELECT INTO** , **INSERT** , **UPDATE** , **DELETE** ) are executed.

2. **Explicit Cursor:** User-defined cursor used when a `SELECT` query is expected to return **more than one row**, giving the developer better control over the result set.

**Implicit Cursor Attributes** (used to check the status of DML operations):

- `%ROWCOUNT` : Returns the number of rows influenced.
- `%NOTFOUND` : Returns `TRUE` if the operation influenced zero rows.
- `%FOUND` : Returns `TRUE` if the operation influenced one or more rows.
- `%ISOPEN` : Returns `FALSE` for implicit cursors because Oracle closes them automatically.

### What is trigger? Types of it?

A **Trigger** is a database object or stored program that is automatically executed (or fired) when a specific event occurs, such as a DML statement ( `INSERT` , `UPDATE` , `DELETE` ) or a DDL statement ( `CREATE` , `DROP` , `ALTER` ). Triggers follow the **Event-Condition-Action (ECA)** model.

Triggers can be classified based on timing and scope:

- **Timing:** They can execute `BEFORE` or `AFTER` the event, or `INSTEAD OF` (for views).
- **Scope:** They can be **row-level** (execute for each row affected, using `FOR EACH ROW` ) or **statement-level** (execute once per SQL statement).

### In which condition we cannot use trigger?

The provided source material describes extensive usage and creation of triggers, including timing ( `BEFORE/AFTER` ) and scope ( `FOR EACH ROW` ), but does not explicitly list conditions or scenarios where a trigger is prohibited or cannot be used.

---

## V. NoSQL and ACID/BASE Properties

### Long form of SQL, NOSQL, RDBMS?

- **SQL:** Structured Query Language.
- **NoSQL:** Not only SQL.
- **RDBMS:** Relational Database Management System.

### Long form of ACID property?

The ACID properties guarantee transaction reliability:

- **A: Atomicity.**
- **C: Consistency.**



- **I: Isolation.**
- **D: Durability.**

### Long form CAP theorem, BASE property?

- **CAP Theorem (Brewer's theorem):** States that a distributed data store cannot simultaneously guarantee:
  - **C: Consistency** (all readers see the latest write).
  - **A: Availability** (system always responds).
  - **P: Partition Tolerance** (system operates despite network failures) [701, 741, Q.5].
- **BASE Property:** Properties favored by many NoSQL systems:
  - **B: Basically Available** (guaranteed to be available in the event of failure).
  - **S: Soft State** (system state may change even without input).
  - **E: Eventual Consistency** (system consistency achieved over time).

### What is the difference between SQL & NoSQL?

| RDBMS (SQL)                                             | NoSQL                                                       |
|---------------------------------------------------------|-------------------------------------------------------------|
| Based on relationships among tables (Relational Model). | Non-relational (Document, Key-value, Graph, Column stores). |
| Uses SQL to query the database.                         | Uses unstructured query language.                           |
| Emphasizes <b>ACID</b> properties.                      | Follows <b>BASE</b> properties and the CAP theorem.         |
| Has predefined, rigid schemas.                          | Schema-free or has a relaxed, dynamic schema.               |
| Vertically scalable.                                    | Horizontally scalable (via sharding/replication).           |

Analogously, if traditional RDBMS (SQL) is like a meticulously organized library with strict rules on where every single document must be filed (ACID), **NoSQL** is like a collection of specialized storage units—some storing simple pairs (key-value), others storing flexible binders (documents), and still others mapping relationships (graphs)—prioritizing the ability to access data quickly even if the synchronization across all units takes a moment (BASE and horizontal scalability).